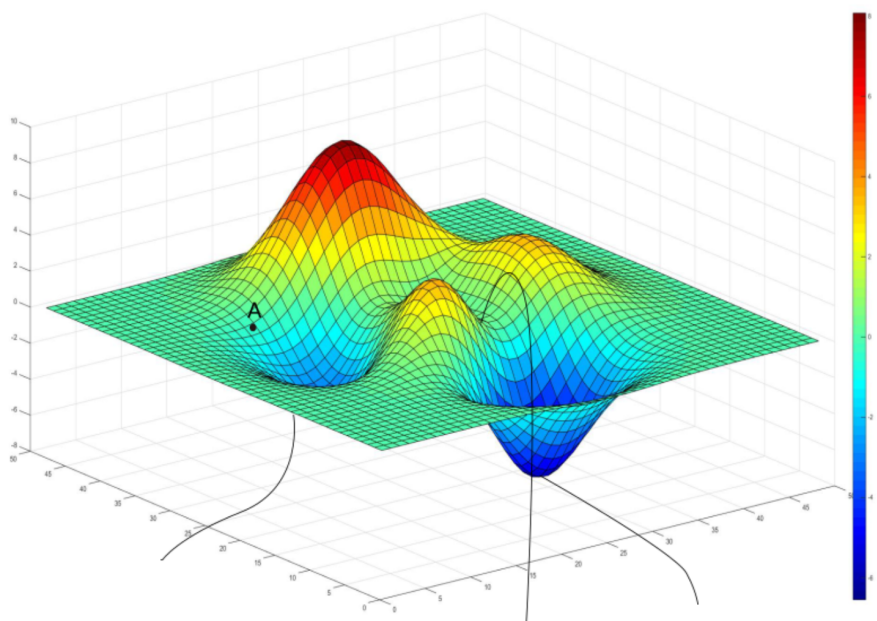

Projet 15

DESCENTE DE GRADIENT ET APPRENTISSAGE PROFOND



Sacha GARROUSTE Thomas MAULINE Jeanne REBERGA
Thomas STAPELFELD Albert TELLIA
Tuteur : Florian BERTUOL

13 avril 2026

■ REMERCIEMENTS

Nous tenons en premier lieu à exprimer notre gratitude à notre tuteur de projet, Monsieur Florian Bertuol, pour son encadrement rigoureux. Ses recommandations quant aux axes de développement à privilégier et la validation régulière de nos avancées ont été des éléments déterminants dans l'aboutissement de ce travail.

Nous remercions également l'équipe pédagogique de la prépa t² pour nous avoir fourni l'opportunité de mener à bien ce projet d'étude, qui a constitué une étape formatrice majeure dans notre cursus d'ingénieur.

Enfin, nous tenons à saluer les chercheurs et vulgarisateurs dont les travaux ont fondamentalement nourri notre réflexion. Nous adressons un hommage particulier à Yann LeCun et Yoshua Bengio, dont les recherches sur l'apprentissage par descente de gradient ont posé les bases de notre étude. Nos remerciements vont aussi à Sebastian Ruder sur les algorithmes d'optimisation, ainsi qu'à Grant Sanderson (3Blue1Brown) et ses ressources pédagogiques vidéo.

Abstract

Artificial intelligence and machine learning currently drive major technological advancements across numerous industrial sectors. However, the increasing complexity of deep learning models introduces significant challenges regarding computational efficiency and mathematical transparency. To address the opacity of modern architectures, it is necessary to examine the fundamental mechanisms of artificial neural networks. The objective of this study is to elucidate the mathematical foundations and the operational architecture of a standard multilayer perceptron.

This research involves the complete development and training of a neural network model, entirely implemented in Python. The methodology relies on the implementation of the gradient descent algorithm and the backpropagation technique to optimize the network weights. To evaluate the theoretical model, a computational implementation was conducted using optimized matrix operations. The experimental phase utilized the EMNIST database, a standard benchmark consisting of handwritten digits and letters, to train and test the character recognition capabilities of the network.

The evaluation of the implemented multilayer perceptron indicates a high level of performance. The model achieved a classification accuracy exceeding 90 percent and computational optimizations resulted in substantially reduced training times.

These findings indicate that an optimized mathematical architecture can execute complex classification tasks efficiently. The results emphasize the importance of fundamental algorithmic optimization in reducing the computational cost of machine learning. Ultimately, this work provides a transparent, foundational model that facilitates a deeper comprehension of complex deep learning systems.

Keywords: *Artificial Intelligence, Multilayer Perceptron, Gradient descent, Character recognition, EMNIST, Optimization, Neural network*

Introduction	4
1 Algorithme de descente de gradient	5
1.1 Explication mathématique de la descente de gradient	5
1.2 Explication de la régression linéaire	5
2 Qu'est-ce qu'un réseau neuronal ?	7
2.1 Modèle d'un neurone	7
2.2 Fonction d'activation	8
2.2.1 Définition	8
2.2.2 Quelques exemples de fonctions d'activations usuelles	8
2.2.3 Fonction Softmax	8
2.3 Architecture du réseau	9
2.3.1 Représentation vectorielle et matricielle	9
2.3.2 Généralisation aux réseaux multicouches	10
2.3.3 Initialisation des paramètres	10
3 Comment les réseaux de neurones apprennent-ils ?	11
3.1 Base de données d'entraînement (EMNIST)	11
3.2 Descente de gradient : (différents types et optimisation)	11
3.2.1 Fonction de coût	11
3.2.2 Différents types de descentes de gradient	12
3.2.3 Comparaison et résultats expérimentaux	12
3.3 Rétropropagation	13
3.3.1 Explication du fonctionnement	13
3.3.2 Propagation de l'erreur vers les couches antérieures	13
3.3.3 Règle de la chaîne et calcul des dérivées pertinentes	13
3.4 Optimisation des paramètres d'entraînement	14
3.4.1 Optimisation du taux d'apprentissage (<i>learning rate</i>)	14
3.4.2 Optimisation via la méthode du <i>Moment</i>	15
3.4.3 Optimisation de la structure du réseau de neurones	15
4 Optimisation Algorithmique et Matérielle	16
4.1 Optimisations structurelles et matérielles	16
4.2 Optimisations spécifiques des opérations mathématiques	17
4.3 Optimisations potentielles et perspectives	17
5 Instant Scénario : De la photo aux neurones	19
5.1 Acquisition (Bibliothèque PIL)	19
5.2 Lissage Gaussien	19

5.3	Rognage Dynamique	19
5.4	Binarisation : La méthode d'Otsu	19
5.5	Normalisation topologique (Padding)	19
5.6	Formatage tensoriel 28x28	19
6	Enjeux socio-écologiques	20
7	Réflexivité, Intelligence collective	21
	Conclusion	22
	Annexes	23

Contexte

En 1943, Warren McCulloch et Walter Pitts proposent le premier modèle mathématique et informatique du neurone biologique [13], en s'appuyant sur les propriétés des neurones biologiques connus à cette époque. En faisant l'analogie avec le cerveau humain, ils publient à la fin des années 1950 des travaux sur le neurone artificiel. C'est ainsi que le psychologue Frank Rosenblatt propose en 1958 le modèle de perceptron, un réseau de neurones multicouches capable « d'apprendre par expérience » sur l'idée de « coefficients synaptiques » capables de s'auto-adapter en s'appuyant sur le calcul du gradient. Ce n'est qu'entre 1970 et 1974 que le concept de rétro-propagation du gradient fut formalisé par Seppo Linnainmaa ; son application aux perceptrons multicouches ouvrit alors la voie à un entraînement plus efficace de ces réseaux.

Périmètre de l'étude

La direction choisie pour ce projet est l'étude et l'implémentation d'un algorithme de descente de gradient sur un perceptron multicouche, appliqué à l'apprentissage supervisé. Bien qu'il existe diverses méthodes d'optimisation pour entraîner les réseaux de neurones (telles que les algorithmes génétiques ou l'optimisation par essaim particulaire), l'approche retenue s'appuie exclusivement sur la descente de gradient, en raison de son rôle fondamental et incontournable dans l'apprentissage profond contemporain. Par ailleurs, afin d'appréhender avec rigueur la complexité des mécanismes mathématiques et algorithmiques sous-jacents, l'intégralité du modèle ainsi que de l'algorithme ont été développés en Python en s'affranchissant délibérément des bibliothèques de haut niveau (telles que TensorFlow ou PyTorch). Le modèle a ainsi été développé intégralement à partir de zéro, en privilégiant l'usage strict d'opérations matricielles élémentaires.

Problématique

Au regard de ce contexte, le présent rapport s'articule autour de la problématique suivante : *Comment implémenter mathématiquement la descente de gradient et optimiser informatiquement un algorithme d'apprentissage profond pour la reconnaissance de caractères manuscrits ?*

Annonce du plan

Pour répondre à cette question, le développement se base autour de quatre axes. Dans un premier temps, nous poserons les fondements mathématiques de la descente de gradient et l'architecture structurelle d'un réseau de neurones. Ensuite, nous détaillerons la mécanique d'apprentissage. Nous analyserons par la suite les optimisations indispensables à l'efficacité du programme. Enfin, nous illustrerons l'application pratique de ces concepts à travers le scénario.

Code source

Le code source de l'algorithme de reconnaissance de caractères manuscrits est à retrouver à l'adresse suivante : [Projet Descente de Gradient et Deep Learning](#)
Le lecteur est invité à le consulter afin de le mettre en perspective avec les notions abordées dans ce rapport.

CHAPITRE 1

ALGORITHME DE DESCENTE DE GRADIENT

1.1. EXPLICATION MATHÉMATIQUE DE LA DESCENTE DE GRADIENT

La descente de gradient est un algorithme d'optimisation itératif fondamental en apprentissage automatique. Son objectif est de trouver le minimum global (ou un minimum local satisfaisant) d'une fonction différentiable, généralement appelée fonction de coût ou fonction de perte, notée $J(\theta) : \mathbb{R}^n \rightarrow \mathbb{R}$, où θ représente le vecteur des paramètres du modèle.

Le principe du gradient

Mathématiquement, le gradient d'une fonction à plusieurs variables, noté $\nabla J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_n} \right)$, est le vecteur composé de l'ensemble de ses dérivées partielles. Une propriété fondamentale de l'analyse vectorielle établit que le gradient pointe toujours dans la direction de la plus forte pente ascendante de la fonction.

Puisque l'objectif de l'apprentissage est de minimiser l'erreur $J(\theta)$, l'algorithme propose de faire évoluer les paramètres dans la direction mathématiquement opposée au gradient.

La règle de mise à jour

L'algorithme procède par itérations successives. À chaque étape n , les paramètres du modèle sont mis à jour selon la relation de récurrence suivante :

$$\theta_{n+1} = \theta_n - \alpha \nabla J(\theta_n)$$

Dans cette équation, le scalaire α représente le taux d'apprentissage (*learning rate*). Pour la descente de gradient classique, il s'agit d'un hyperparamètre fixe défini avant l'entraînement. Il est toutefois important de noter que des variantes plus sophistiquées (détaillées en section 3.4.1) utilisent des pas adaptatifs qui s'ajustent dynamiquement en fonction de la géométrie de la fonction de coût.

- Si α est trop petit, la convergence vers le minimum sera extrêmement lente, nécessitant un temps de calcul prohibitif.
- Si α est trop grand, l'algorithme risque de dépasser le minimum et de diverger, rendant l'apprentissage impossible.

1.2. EXPLICATION DE LA RÉGRESSION LINÉAIRE

La régression linéaire constitue l'application la plus élémentaire de la descente de gradient. Elle permet de modéliser la relation entre une variable explicative x (l'entrée) et une variable à prédire y (la sortie). Ce modèle est conceptuellement l'ancêtre d'un neurone artificiel unique dépourvu de fonction d'activation non linéaire.

Modélisation mathématique

L'hypothèse de la régression linéaire repose sur l'équation d'une droite :

$$\hat{y} = wx + b$$

Où :

- $\hat{y}$ est la prédiction générée par le modèle.
- w (le poids, ou coefficient directeur) détermine la pente.
- b (le biais, ou ordonnée à l'origine) décale la droite.

La fonction de coût

Pour évaluer la précision de cette droite par rapport à un jeu de données $\mathcal{D}$ réel constitué d'un ensemble de $\text{card}(\mathcal{D})$ points (x_i, y_i) , il convient de mesurer l'erreur entre les prédictions et la réalité. L'approche standard utilise l'Erreur Quadratique Moyenne (MSE - Mean Squared Error) :

$$J(w, b) = \frac{1}{\text{card}(\mathcal{D})} \sum_{i \in \mathcal{D}} (\hat{y}_i - y_i)^2$$

où $\hat{y}_i = wx_i + b$

Cette fonction calcule la moyenne des carrés des distances verticales entre chaque point de donnée et la droite de régression. L'élévation au carré garantit que les erreurs positives et négatives ne s'annulent pas et pénalise plus fortement les grandes erreurs.

Application de la descente de gradient

Afin de trouver la droite optimale, il faut minimiser $J(w, b)$. L'algorithme calcule les dérivées partielles de la fonction de coût par rapport à chacun des paramètres :

Dérivée par rapport au poids w :

$$\frac{\partial J}{\partial w} = \frac{2}{N} \sum_{i=1}^N x_i (\hat{y}_i - y_i)$$

Dérivée par rapport au biais b :

$$\frac{\partial J}{\partial b} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i)$$

Les paramètres sont ensuite simultanément mis à jour à chaque itération jusqu'à convergence :

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$
$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

Cette méthode itérative garantit, par exemple, dans le cas d'une régression linéaire dont la fonction de coût est strictement convexe, de trouver l'unique minimum global de la fonction.

CHAPITRE 2

QU'EST-CE QU'UN RÉSEAU NEURONAL?

La reconnaissance de formes, telle que l'identification de caractères manuscrits, est une tâche naturelle et intuitive pour le système visuel humain mais complexe pour une machine. Dans le cadre de la base de données EMNIST (cf. 3.1), chaque échantillon est une image de 28×28 pixels. Pour un ordinateur, cette image n'est qu'une matrice de 784 valeurs numériques.

L'objectif d'un réseau de neurones est de transformer ce vecteur de données brutes en une information sémantique (le chiffre de 0 à 9 ou lettre de A à Z). Nous allons ici détailler la structure de cet objet mathématique. Ce chapitre tire ses sources de l'article « *Réseaux de neurones* » de Marc Parizeau. [10]

2.1. MODÈLE D'UN NEURONE

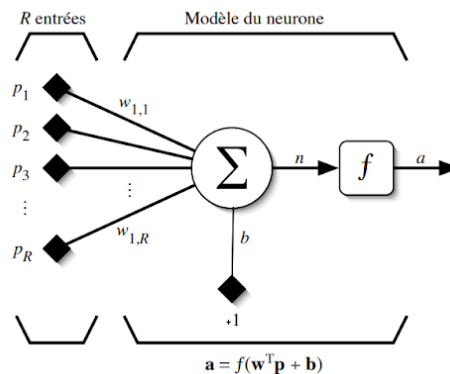


FIGURE 2.1 – Représentation mathématique d'un neurone artificiel

Un neurone est essentiellement constitué d'un intégrateur qui effectue la somme pondérée de ses entrées. Le résultat n de cette somme est ensuite transformé par une fonction de transfert f qui produit la sortie a du neurone. Les R entrées du neurone correspondent au vecteur $\mathbf{P} = [p_1, p_2, \dots, p_R]^T$, alors que $\mathbf{w} = [w_{1,1}, w_{1,2}, \dots, w_{1,R}]^T$ représente le vecteur des poids du neurone. La sortie n de l'intégrateur est donnée par l'équation suivante :

$$n = \sum_{j=1}^R w_{1,j} p_j + b = w_{1,1} p_1 + w_{1,2} p_2 + \dots + w_{1,R} p_R + b$$

que l'on peut aussi écrire sous forme matricielle :

$$n = \mathbf{w}^T \mathbf{p} + b$$

Cette sortie correspond à une somme pondérée des poids et des entrées plus ce qu'on nomme le biais b du neurone. Le résultat n de la somme pondérée s'appelle **le niveau d'activation du neurone**. Le biais b s'appelle aussi **le seuil d'activation du neurone**. Lorsque le niveau d'activation atteint ou dépasse le seuil b , alors l'argument de f devient positif (ou nul). Sinon, il est négatif.

2.2. FONCTION D'ACTIVATION

2.2.1. Définition

La **fonction d'activation**, notée f ou σ dans le modèle du neurone, joue un rôle déterminant dans la capacité d'apprentissage d'un réseau profond. Son rôle principal est d'introduire une non-linéarité dans le calcul de la propagation avant (*forward pass*). Sans elles, une succession de couches se réduirait mathématiquement à une simple combinaison linéaire, équivalente à une régression classique incapable de traiter la complexité de la base EMNIST. Ainsi, elles transforment le signal d'entrée d'un nœud d'un réseau neuronal en un signal de sortie qui est ensuite transmis à la couche suivante.

2.2.2. Quelques exemples de fonctions d'activations usuelles

Fonction d'activation Sigmoid :

La fonction sigmoïde est une équation mathématique qui associe tout nombre réel à une valeur comprise entre 0 et 1. Sa formule est donnée ci-dessous :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

La fonction sigmoïde présente des limites, la principale étant le problème du gradient de fuite. Pour des valeurs d'entrée très grandes ou très petites, la sortie de la fonction sature près de 1 ou de 0, et son gradient devient presque nul. Cela entraîne un ralentissement du processus d'apprentissage dans les réseaux neuronaux denses, car les poids sont mis à jour trop lentement au cours de la formation. Pour remédier à cette limitation, d'autres fonctions d'activation telles que ReLU (Rectified Linear Unit) et Tanh sont souvent utilisées.

Fonction ReLU (Rectified Linear Unit) :

La fonction **ReLU** est définie par : $\text{ReLU}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$. Elle est devenue la norme pour les couches cachées grâce à trois avantages majeurs. Contrairement à la sigmoïde, la **ReLU** ne sature pas pour les valeurs positives. Son gradient reste constant, ce qui empêche la disparition du signal d'erreur lors de la rétropropagation et permet d'entraîner des réseaux très profonds. En annulant les sorties pour toute entrée négative, elle désactive une partie des neurones. Cette "rareté" des activations produit des représentations plus efficaces et facilite l'extraction de caractéristiques pertinentes. Sa simplicité mathématique $\max(\mathbf{0}, \mathbf{x})$ réduit drastiquement le coût de calcul par rapport aux fonctions exponentielles, accélérant ainsi la phase d'apprentissage.

2.2.3. Fonction Softmax

La fonction softmax est une fonction non linéaire utilisée en couche de sortie pour les problèmes de classification multiclasse. Elle transforme un vecteur de scores réels en une distribution de probabilités.

$$F(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad j \in \{1, \dots, K\}$$

où $z \in \mathbb{R}^K$ est le vecteur des scores en entrée, et $F(z)_j$ représente la probabilité associée à la sortie j .

Chaque sortie j correspond à une classe, c'est-à-dire une catégorie possible prédite par le modèle (par exemple, les chiffres de 0 à 9 en reconnaissance de chiffres manuscrits).

La fonction softmax et la fonction sigmoïde partagent une même base mathématique, car elles reposent toutes deux sur l'utilisation de l'exponentielle. Contrairement à la sigmoïde, elle introduit une dépendance entre les sorties. Comme la somme des probabilités doit être égale à 1 (100%), si la probabilité d'une classe augmente, les autres diminuent mécaniquement. Cela force le réseau à faire un choix plus tranché entre les chiffres.

2.3. ARCHITECTURE DU RÉSEAU

2.3.1. Représentation vectorielle et matricielle

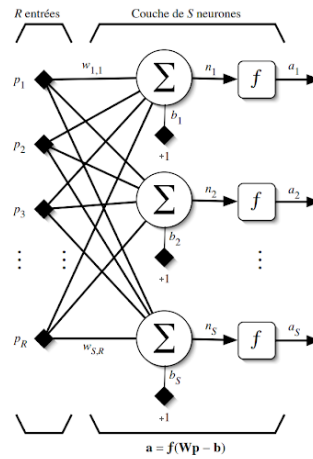


FIGURE 2.2 – Représentation réseau de neurone

Un réseau de neurones est un maillage de plusieurs neurones, généralement organisé en couches. Pour construire une couche de S neurones, il s'agit simplement de les assembler comme sur la figure 2.2. Les S neurones d'une même couche sont tous branchés aux R entrées. On dit alors que la couche est totalement connectée. Un poids $w_{i,j}$ est associé à chacune des connexions. Nous noterons toujours le premier indice par i et le deuxième par j (jamais l'inverse). Le premier indice (ligne) désigne toujours le numéro de neurone sur la couche, alors que le deuxième indice (colonne) spécifie le numéro de l'entrée. Ainsi, $w_{i,j}$ désigne le poids de la connexion qui relie le neurone i à son entrée j . L'ensemble des poids d'une couche forme donc une matrice $\mathbf{W}$ de dimension $S \times R$:

$$\mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & \ddots & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$

FIGURE 2.3 – Matrice des poids du réseau

Pour une couche donnée comportant S neurones recevant R entrées, les dimensions sont indépendantes.

Pour traiter une couche globalement, nous regroupons les paramètres individuels sous forme de vecteurs et de matrices. Soient :

- $\mathbf{b} = (b_1, b_2, \dots, b_S)^T$: le vecteur des biais ;
- $\mathbf{n} = (n_1, n_2, \dots, n_S)^T$: le vecteur des sommes pondérées (niveaux d'activation) ;
- $\mathbf{a} = (a_1, a_2, \dots, a_S)^T$: le vecteur des sorties (activations) ;
- $\mathbf{W} \in \mathbb{R}^{S \times R}$: la matrice des poids ;
- $\mathbf{p} \in \mathbb{R}^R$: le vecteur d'entrée.

L'opération de la couche complète se résume par les équations matricielles :

$$\mathbf{n} = \mathbf{W}\mathbf{p} + \mathbf{b}$$

$$\mathbf{a} = F(\mathbf{n})$$

où F est appliquée composante par composante :

$$F(\mathbf{n}) = (f(n_1), \dots, f(n_S))^T$$

2.3.2. Généralisation aux réseaux multicouches

Un réseau profond est constitué par la cascade de plusieurs couches. La sortie d'une couche $k - 1$ devient l'entrée de la couche k . Nous adoptons la notation en exposant pour désigner l'indice de la couche :

- S^k : nombre de neurones de la couche k .
- $\mathbf{W}^k$: matrice des poids de la couche k .
- f^k : fonction de transfert de la couche k .

Ainsi, pour un réseau à M couches, le transfert d'information suit la relation de récurrence :

$$\mathbf{a}^k = f^k \left(\mathbf{W}^k \mathbf{a}^{k-1} + \mathbf{b}^k \right)$$

2.3.3. Initialisation des paramètres

Avant que le processus d'apprentissage ne débute, les paramètres du réseau ($\mathbf{W}$ et $\mathbf{b}$) doivent être initialisés. Cette étape, souvent invisible mais fondamentale, conditionne la convergence du modèle :

- **Le problème de la symétrie** : On ne peut pas initialiser tous les poids à zéro. Si tous les neurones d'une couche partagent les mêmes poids initiaux, ils recevront le même gradient et subiront exactement la même mise à jour. Ils resteront donc identiques, rendant inutile la multiplication des neurones. C'est ce qu'on appelle la « rupture de symétrie ».

- **Stratégies utilisées** : Les poids sont généralement initialisés de manière aléatoire suivant des lois de probabilité spécifiques. Des méthodes comme l'initialisation de Glorot (Xavier) ou de He ajustent la variance des poids en fonction du nombre d'entrées (R) et de sorties (S) pour éviter que le signal ne s'estompe ou n'explose à travers les couches.

CHAPITRE 3

COMMENT LES RÉSEAUX DE NEURONES APPRENNENT-ILS ?

Une fois le réseau de neurones modélisé (nombre de neurones et couches), il convient donc de l'entraîner sur un large spectre de donnée. Pour cela on utilise l'algorithme de descente de gradient décrit plus tôt couplé à une méthode de calcul de gradient propre à ce type de réseau neuronaux appelé rétropropagation.

Le type d'entraînement décrit ici est dit « **supervisé** » car chaque donnée est étiquetée. La fonction d'erreur se base sur ce principe. Mais il existe d'autres types d'apprentissage (non supervisé, par renforcement), non décrits dans ce rapport.

3.1. BASE DE DONNÉES D'ENTRAÎNEMENT (EMNIST)

Le but du réseau de neurones décrit dans ce rapport étant la **reconnaissance de caractères manuscrits**, une base de données d'entraînements est donc indispensable pour entraîner le modèle convenablement. Le choix s'est porté sur une base de données fondamentale dans le domaine du deep learning : la base de données EMNIST (Extended Modified NIST).

Celle-ci est issue d'une base plus large appelée NIST Special database 19 (1995) provenant de l'agence américaine National Institute of Standards and Technology (NIST). D'après la documentation officielle associée [3], celle-ci comprend environ 700 000 images pixelisées de 28x28 en niveau de gris de **chiffres, lettres majuscules et lettres minuscules** manuscrites écrites par plus de 500 scripteurs. À Chaque donnée est attribuée **une étiquette** décrivant l'image (« A » ou « k »)

Cette base de données offre un large choix de segmentations. Par exemple, il est possible de ne sélectionner que les chiffres, que les lettres ou les deux simultanément. Ce qui change, c'est le nombre des « **classes** » différentes (catégorie regroupant des données partageant des caractéristiques communes, un même étiquette). Le nombre de classes définit aussi la taille de la dernière couche de neurones du réseau. (voir la segmentation complète sur la figure 3.1)

3.2. DESCENTE DE GRADIENT : (DIFFÉRENTS TYPES ET OPTIMISATION)

Cette section décrit les différents types de descente de gradient explorés. Ils se basent tous sur le même principe mais possèdent des différences qui impactent de manière non négligeable les performances d'apprentissage du réseau de neurone.

3.2.1. Fonction de coût

La fonction de coût reste l'erreur quadratique moyenne (cf. 1.2), à ceci près que la prédiction $\hat{y}$ est désormais une matrice colonne — de taille égale au nombre de classes — contenant les activations de la couche de sortie. L'étiquette y est une matrice colonne de même taille, avec un 1 sur la classe cible et des 0 partout ailleurs (encodage *one-hot*).

3.2.2. Différents types de descentes de gradient

Descente de gradient Batch (globale)

Étant donné que la fonction d'erreur présentée ci-dessus calcule l'erreur sur un jeu de données défini $\mathcal{D}$, la méthode de descente de gradient qui vient le plus naturellement est celle utilisant le calcul du gradient de cette même fonction.

Cette méthode de descente de gradient est appelé *Batch*. Cette notation venant de l'anglais signifie « lot ». À chaque étape de l'algorithme, le gradient calculé est celui de la fonction d'erreur qui dépend de l'intégralité des données contenues dans le **lot de données d'entraînement**.

Descente de gradient Stochastique (Unitaire)

Une alternative d'optimisation est la descente de gradient dite *stochastique*. Cette méthode consiste à calculer le gradient de l'erreur d'une seule donnée aléatoire du jeu à chaque itération (on change $\mathcal{D}$ par $d \in \mathcal{D}$). Le gradient calculé n'est alors qu'une estimation du réel gradient de la fonction d'erreur ce qui implique une courbe d'apprentissage très bruitée.

Descente de gradient par Mini-Batch

Enfin, une méthode de descente intermédiaire peut être utilisée. Ce compromis entre les deux approches précédentes est appelé descente par *mini-batch*. Cet algorithme consiste à ne pas calculer le gradient de la fonction d'erreur sur l'ensemble $\mathcal{D}$ entier, mais sur des petits lots de données $\mathcal{B} \subset \mathcal{D}$ composés d'un nombre fixe de données. En réalité, cet algorithme appartient aussi à la famille des *stochastiques*, mais nous réserverons ce terme pour l'algorithme unitaire décrit ci-dessus.

3.2.3. Comparaison et résultats expérimentaux

Batch	Stochastique	Mini-batch
while epoques $\leq n$ do $G \leftarrow \text{grad}(\text{batch})$ poids $\leftarrow$ poids $- \alpha \times G$	while epoques $\leq n$ do mélanger les données for donnée in batch do $G \leftarrow \text{grad}(\text{donnée})$ poids $\leftarrow$ poids $- \alpha \times G$	while epoques $\leq n$ do mélanger les données découper en mini-batch for mini in batch do $G \leftarrow \text{grad}(\text{mini})$ poids $\leftarrow$ poids $- \alpha \times G$

Pour comparer ces méthodes, nous utilisons la notion d'Époque. Une époque correspond à un passage complet de l'algorithme sur l'intégralité du jeu de données. C'est notre étalon pour mesurer la vitesse d'apprentissage.

Par époque, *batch* n'effectue qu'une seule mise à jour des poids contre n pour *stochastique*. Cela explique que *stochastique* converge plus vite et atteint de meilleurs minima locaux — le bruit dans les mises à jour agissant comme un mécanisme d'exploration (3.2, 3.3 et 3.4) [7]. En revanche, ce surplus de calculs de gradients devient prohibitif sur de grands jeux de données (EMNIST $\approx 100\,000$ exemples), ralentissant considérablement l'entraînement.

Mini-batch réalise un compromis : une vitesse de convergence satisfaisante (figure 3.4) pour un coût calculatoire raisonnable (figure 3.5), ce que confirme expérimentalement la comparaison des

rapports temps/taux de réussite (figure 3.6).

3.3. RÉTROPROPAGATION

3.3.1. Explication du fonctionnement

La rétro-propagation est une méthode de calcul d'un seul gradient qui réalise le chemin inverse du *FeedForward* :

La couche de sortie propage les erreurs obtenues vers les couches précédentes. Les neurones de sortie, notés comme la couche **M**, présentent une erreur locale. Celle-ci est provoquée principalement par les neurones connectés par un poids très grand, et l'algorithme va donc propager l'erreur locale de chaque neurone de la couche **M** vers la couche (**M** - 1) en la multipliant par le poids associé, attribuant une valeur représentant la responsabilité du neurone de la couche (**M** - 1) dans l'erreur de la couche **M**.

3.3.2. Propagation de l'erreur vers les couches antérieures

En suivant le principe énoncé ci-dessus, nous pouvons établir une formule mathématiques qui calcule le blâme de chaque neurone précédent, noté $\delta_j^{(M)}$, quantifiant à quel point le neurone précédent est responsable dans l'erreur $\delta_i^{(M)}$ des neurones de sortie, le tout multiplié par la dérivée de la fonction d'activation σ appliqué au neurone de la couche (**M** - 1) qui caractérise à quel point ce neurone est « sûr » de son résultat (et donc s'il peut changer). Cela est modélisé par la relation suivante :

$$\delta_j^{(M-1)} = \sum_{i=1}^{S^{(M)}} (\delta_i^{(M)} \cdot w_{i,j}^{(M)}) \cdot \sigma'(n_j^{(M-1)})$$

(Cette notion nous sera utile pour comprendre la suite et sera retrouvée dans les calculs.)

3.3.3. Règle de la chaîne et calcul des dérivées pertinentes

Règle de la chaîne et définitions

$$\frac{\partial E}{\partial w_{jk}^{(M)}} = \frac{\partial E}{\partial a_j^{(M)}} \cdot \frac{\partial a_j^{(M)}}{\partial n_j^{(M)}} \cdot \frac{\partial n_j^{(M)}}{\partial w_{jk}^{(M)}}$$

- E l'erreur quadratique
- w_{jk} le poids (reliant couche (M) et (M-1))
- a_j l'activation du neurone j couche (M)
- n_j la somme pondérée des activations et poids du neurone j

Le calcul explicite de chacune de ces dérivées partielles permet d'obtenir :

$$(1) \quad \frac{\partial E}{\partial a_j^{(M)}} = \frac{\partial \frac{1}{n} \sum_{i=1}^n (a_i^{(M)} - y_i^{(M)})^2}{\partial a_j^{(M)}} = \frac{2}{n} (a_j^{(M)} - y_j^{(M)})$$

$$(2) \quad \frac{\partial a_j^{(M)}}{\partial n_j^{(M)}} = \frac{\partial \sigma_j(n_j^{(M)})}{\partial n_j^{(M)}} = \sigma'_j(n_j^{(M)})$$

$$(3) \quad \frac{\partial n_j^{(M)}}{\partial w_{jk}^{(M)}} = a_k^{(M-1)}$$

Avec σ la fonction d'activation et k l'indice d'un neurone de la couche précédente.

Que signifient les dérivées ?

On remarque que dans les différentes dérivées :

$$\frac{\partial E}{\partial w_{jk}^{(M)}} = \underbrace{\frac{\partial E}{\partial a_j^{(M)}} \cdot \frac{\partial a_j^{(M)}}{\partial n_j^{(M)}}}_{\delta_j^{(M)}} \cdot \underbrace{\frac{\partial n_j^{(M)}}{\partial w_{jk}^{(M)}}}_{a_k^{(M-1)}}$$

On peut ainsi répéter les étapes entre les couches :

$$\begin{aligned} \frac{\partial E}{\partial n_j^{(M-1)}} &= \frac{\partial E}{\partial a_j^{(M-1)}} \cdot \frac{\partial a_j^{(M-1)}}{\partial n_j^{(M-1)}} = \sum_{k=1} \left(\frac{\partial E}{\partial n_k^{(M)}} \cdot \frac{\partial n_k^{(M)}}{\partial a_j^{(M-1)}} \right) \cdot \sigma'_j(n_j^{(M-1)}) \\ &= \sum_{k=1} \left(\delta_k^{(M)} \cdot w_{kj}^{(M)} \right) \cdot \sigma'_j(n_j^{(M-1)}) = \delta_j^{(M-1)} \end{aligned}$$

Ce qui donnerait que :

$$\frac{\partial E}{\partial w_{jk}^{(M-1)}} = \underbrace{\frac{\partial E}{\partial a_j^{(M-1)}} \cdot \frac{\partial a_j^{(M-1)}}{\partial n_j^{(M-1)}}}_{\delta_j^{(M-1)}} \cdot \underbrace{\frac{\partial n_j^{(M-1)}}{\partial w_{jk}^{(M-1)}}}_{a_k^{(M-2)}} \quad (*)$$

Le calcul du gradient de la fonction d'erreur est donc fait une seule fois sans avoir à recalculer un taux de variation par rapport à chaque poids. Les poids seront modifiés par l'algorithme de descente de gradient dont les dérivées partielles dépendent donc du produit du blâme et de la dérivée de la fonction d'activation (voir (*)). Donc chaque vecteur d'erreur permet de calculer le vecteur d'erreur de la couche précédente, et cela permet donc de calculer rapidement chaque dérivée partielle, donc d'optimiser le calcul du gradient.

3.4. OPTIMISATION DES PARAMÈTRES D'ENTRAÎNEMENT

Après avoir sélectionné la méthode de descente de gradient optimale (mini-batch) combinée avec la méthode de la rétropropagation, il convient d'étudier l'effet des autres paramètres sur l'efficacité du réseau. Cette étude a été réalisée sur deux jeux de données en parallèle : « *digits* » qui contient seulement les 10 chiffres et « *balanced* » qui contient les chiffres et les lettres avec une répartition égale de tous les caractères.

3.4.1. Optimisation du taux d'apprentissage (*learning rate*)

Le taux d'apprentissage (*learning rate*) ou « pas » souvent noté α est un paramètre crucial car il intervient directement dans le calcul de mise à jour des poids (cf. 1.1).

En faisant varier celui-ci on voit qu'il est nécessaire d'optimiser sa valeur. Si elle est trop petite, l'apprentissage devient très lent et le minimum local de la fonction d'erreur n'est pas atteint. À l'inverse, si celle-ci est trop élevée, l'apprentissage devient trop agressif ce qui fait que les pas ne sont pas assez précis pour atteindre le minimum local.

Les figures 3.7 et 3.8 illustrent ces phénomènes.

De nos jours il existe des méthodes dites de « pas adaptatif » qui permettent d'ajuster, lors

de la descente de gradient, le pas de chaque poids individuellement. Ces méthodes se basent notamment sur des méthodes de calcul différentiel incluant des calculs de dérivées partielles secondes dans la **rétropropagation**

3.4.2. Optimisation via la méthode du *Moment*

La méthode du *Moment* ou *Moment de Nesterov* sont des méthodes d'optimisation qui interviennent lors de la mise à jour des poids. Le principe est qu'à chaque itération, la mise à jour des poids prend aussi en compte la norme du gradient de l'itération antérieure dans son calcul :

$$\textbf{Moment} : \theta_{n+1} = \theta_n + \mu(\theta_n - \theta_{n-1}) - \alpha \nabla J(\theta_n)$$

$$\textbf{Nesterov} : \theta_{n+1} = \theta_n + \mu(\theta_n - \theta_{n-1}) - \alpha \nabla J(\theta_n + \mu(\theta_n - \theta_{n-1}))$$

Ces méthodes permettent une vitesse de convergence accrue vers les minimums locaux de la fonction de coût. (Voir figure 3.9 et figure 3.10)

3.4.3. Optimisation de la structure du réseau de neurones

Nombre de neurones sur une couche cachée

Enfin la dernière étape de l'optimisation d'un réseau est l'étude de sa forme : son nombre de neurones et nombre de couches cachées. On appelle ce paramètre **la largeur du réseau**. on remarque que plus le nombre de neurones augmente, meilleurs sont les courbes d'apprentissages ainsi que les taux de réussite finaux. (Voir figure 3.11 et figure 3.12 pour « *digit* ») et (Voir figure 3.13 et figure 3.14 pour « *balanced* »).

Cependant on observe déjà un phénomène **d'étranglement de l'information** pour le jeu « *balanced* ». en effet, il y a exactement 47 classes de caractères différents sur la couche finale. Disposer une couche intermédiaire avec moins de neurones que la couche finale forme un goulot d'étranglement de l'information ce qui diminue drastiquement le taux de réussite du réseau.

Nombre de couches cachées

Ajouter des couches cachées augmente théoriquement les capacités du réseau. Cependant, il est encore question du cas du goulot d'étranglement comme on peut le voir sur la figure 3.15. Pour pallier ce problème, il est donc logique d'augmenter le nombre de neurones intermédiaire et d'opter pour une architecture pyramidale pour lisser l'information le long de toutes les couches. Les résultats sont alors très satisfaisants pour le jeu de donnée « *balanced* » (Voir figure 3.16)

Mais il ne faut pas oublier aussi que plus le nombre de neurones devient grand, plus les temps de calculs augmentent. Il est alors intéressant de calculer le rapport suivant :

$$\frac{\text{Taux de réussite}}{\text{Nombre de neurones total}}$$

Dans ce cas, le réseau [512, 256] de la figure 3.17 devient le plus avantageux.

Pour le jeu « *balanced* », les taux de réussites sont moins élevés que « *digits* » en raison de la difficulté accrue de classification (voir exemple d'échec à la figure 3.18 et exemple de succès à la figure 3.19).

L'entraînement d'un perceptron multicouche sur une base de données telle que EMNIST requiert des dizaines de millions de calculs arithmétiques par époque. Le langage Python, étant interprété et non compilé, est intrinsèquement lent pour exécuter des calculs itératifs massifs. Afin de rendre le temps d'apprentissage viable, il est indispensable de restructurer le code pour transférer la charge de calcul du niveau logiciel (Python) vers le niveau matériel (le processeur). Ce chapitre détaille les différentes strates d'optimisation appliquées au projet.

4.1. OPTIMISATIONS STRUCTURELLES ET MATÉRIELLES

Architecture SIMD et bibliothèques BLAS/LAPACK

L'optimisation première consiste à fuir les listes natives de Python. La vectorisation des données sous forme de tableaux NumPy permet de déléguer les calculs à des bibliothèques de bas niveau précompilées (généralement en langage C ou Fortran), telles que BLAS (Basic Linear Algebra Subprograms). Ces standards communiquent directement avec l'architecture physique du processeur pour exploiter le jeu d'instructions SIMD (Single Instruction, Multiple Data).

Concrètement, au lieu de demander au processeur d'additionner deux nombres à la fois lors d'un cycle d'horloge, le SIMD lui permet d'appliquer cette addition simultanément sur un vecteur entier de dizaines de données, accélérant drastiquement la vitesse d'exécution globale.

Broadcasting et traitement par lots (Mini-Batch)

L'approche naïve consisterait à traiter les 131 600 échantillons du dataset de manière séquentielle, générant un coût d'interprétation prohibitif pour chaque image. L'optimisation par « Mini-Batch » regroupe les données par lots de taille $n = 32$, réduisant le nombre d'appels aux fonctions de calcul à environ 4 113 opérations matricielles par époque. Grâce au mécanisme de *Broadcasting* de NumPy, l'équation $\mathbf{N} = \mathbf{WA} + \mathbf{B}$ est traitée par blocs, optimisant l'usage des registres du processeur et répartissant la charge sur l'ensemble des cœurs physiques.

Typage strict et allègement mémoire

Par défaut, Python assigne le type `float64` (flottant sur 64 bits) à tout nombre décimal, offrant une précision extrême (15 décimales) qui est superflue pour la convergence d'un réseau de neurones. Ce typage par défaut sature inutilement la mémoire cache du processeur.

Pour y remédier, les images brutes ont été contraintes au format entier non signé sur 8 bits (`np.uint8`), et les matrices de poids au format `np.float32`. Cette réduction stricte de l'empreinte mémoire fluidifie les transferts d'informations entre la mémoire vive (RAM) et le processeur, doublant mécaniquement la vitesse des multiplications matricielles.

Pré-allocation vs Réallocation dynamique

Un problème courant en Python est l'allocation dynamique : lorsqu'on ajoute une valeur à une liste (avec `.append()`), le système d'exploitation doit régulièrement chercher de nouveaux blocs de mémoire libre, ce qui fige l'exécution. Puisque le nombre total d'itérations d'entraînement est défini à l'avance, l'espace mémoire destiné à recevoir l'historique des erreurs est pré-alloué dès le lancement du programme (via un tableau de zéros). L'enregistrement des pertes se résume alors à insérer une valeur à un indice précis, une opération exécutée en temps constant $\mathcal{O}(1)$.

4.2. OPTIMISATIONS SPÉCIFIQUES DES OPÉRATIONS MATHÉMATIQUES

L'illusion de la fonction `np.vectorize`

Pour appliquer une fonction mathématique complexe (comme la dérivée de la fonction d'activation), il est tentant d'utiliser la commande `np.vectorize`. Or, cette fonction est une illusion d'optimisation : elle se contente d'encapsuler une boucle Python classique pour l'appliquer à chaque élément d'une matrice, sans aucune accélération matérielle. Pour contourner ce goulot d'étranglement, l'implémentation s'appuie sur des masques booléens (comme `np.where`) ou le filtrage direct (`np.clip`). Ces opérations évaluent la matrice comme un bloc unique et sont exécutées de manière native en C, éliminant totalement l'usage des boucles logicielles.

Vectorisation absolue de la rétropropagation

La rétropropagation de l'erreur exige de calculer le gradient pour chaque poids de chaque neurone. Sur un réseau, qui compte plus de $5,4 \times 10^5$ poids synaptiques pour une architecture [512, 256], cela est réalisée via un produit matriciel pur. En substituant les boucles itératives natives de Python par des routines BLAS/LAPACK pré-compilées, cette approche permet un gain de performance d'environ deux ordres de grandeur. La formulation de la règle de la chaîne sous forme de produit de la matrice des erreurs Δ par la transposée des activations $\mathbf{A}^T$ transforme ainsi l'ajustement de centaines de milliers de variables en une instruction matérielle unique.

Préservation de la contiguïté mémoire

Dans un ordinateur, lire des données dispersées dans la mémoire prend beaucoup plus de temps que de lire un bloc continu. Les listes natives de Python (et particulièrement les listes de listes) fragmentent les données. Pour garantir un accès mémoire instantané, toutes les données d'entraînement (images et étiquettes) sont encapsulées et maintenues dans des tenseurs NumPy (`np.ndarray`) de bout en bout du script. Cette contiguïté physique des données dans la mémoire vive permet aussi un découpage (Slicing) instantané des lots d'images lors de l'apprentissage.

4.3. OPTIMISATIONS POTENTIELLES ET PERSPECTIVES

Régularisation Lasso (Pénalité L1)

Afin d'optimiser le modèle lors de son utilisation finale (*inférence*), une pénalité L1 (*Lasso*) pourrait être ajoutée à la fonction de coût. Mathématiquement, cette technique force le réseau à annuler totalement les poids synaptiques qui n'apportent pas d'information pertinente.

Cela génère des matrices dites « creuses » (*Sparse Matrices*), composées majoritairement de zéros. L'avantage algorithmique est majeur : les bibliothèques d'algèbre linéaire ignorent les multiplications par zéro, réduisant ainsi drastiquement le nombre de calculs nécessaires pour faire une prédiction.

Accélération GPU (Architecture CUDA)

Le traitement d'un perceptron reposant quasi-exclusivement sur des opérations matricielles, la migration de l'entraînement vers un processeur graphique (GPU) représente l'optimisation ultime. Là où un processeur central (CPU) possède une dizaine de cœurs optimisés pour la logique séquentielle, un GPU embarque des milliers de cœurs dédiés au calcul matriciel hautement parallélisé. La simple substitution de la bibliothèque NumPy par CuPy permettrait, sans altérer la logique mathématique du code, d'exploiter cette architecture et de diviser le temps de calcul de manière exponentielle.

Synthèse des bonnes pratiques d'optimisation

L'implémentation efficace d'un algorithme d'apprentissage machine impose la validation systématique des critères suivants :

- **Profilage (Profiling)** : Mesurer mathématiquement l'exécution (via `cProfile`) pour identifier les goulots d'étranglement réels avant d'optimiser le code.
- **Éradication des boucles internes** : Supprimer toute itération `for` traversant des données numériques au profit de calculs vectorisés purs.
- **Contiguïté spatiale** : Maintenir les données dans des tableaux dimensionnels de bas niveau pour minimiser le temps d'accès de la mémoire cache.
- **Justesse du typage** : Éviter le surtypage en assignant la taille en octets la plus petite possible (8 ou 32 bits au lieu de 64 bits) selon le besoin de précision.
- **Allocation prédictive** : Réserver l'intégralité des blocs de mémoire requis dès l'initialisation du programme, prohibant ainsi l'allocation dynamique en cours de boucle.

CHAPITRE 5

INSTANT SCÉNARIO : DE LA PHOTO AUX NEURONES

Acquisition (Bibliothèque PIL)

L'ouverture de la photographie brute, la conversion primaire en niveaux de gris et les manipulations topologiques de base sont réalisées via la Python Imaging Library (PIL).

Lissage Gaussien

Une photographie présente systématiquement un bruit de fond, causé par le capteur optique et les algorithmes de compression, qui perturberait l'analyse de contraste. L'image subit un lissage spatial par filtre Gaussien de rayon 1. Contrairement à un filtre moyenneur classique qui dégrade uniformément la netteté, le noyau gaussien permet d'homogénéiser efficacement la texture du papier tout en préservant l'intégrité et la topologie des contours du trait d'encre.

Rognage Dynamique

L'éclairage d'une photographie étant rarement parfait, les bordures présentent souvent des ombres portées qui fausseraient la binarisation globale. Un algorithme de balayage sous NumPy identifie les coordonnées extrêmes de la trace d'encre permettant d'éliminer les marges inutiles. Afin de pallier l'imprévisibilité de l'exposition, le système est interactif : il génère simultanément trois recadrages basés sur des seuils d'intensité progressifs (ex : 60, 70 et 80). L'utilisateur peut ainsi sélectionner visuellement l'extraction la plus pertinente. (Voir fonction `choisir_rognage` en annexe).

Binarisation : La méthode d'Otsu

L'objectif est d'isoler définitivement la structure utile en transformant la matrice en niveaux de gris (0 à 255) en un format strictement binaire (0 ou 1). L'application d'un seuil de coupure fixe échouerait face aux variations de luminosité. L'algorithme d'Otsu, implémenté de manière vectorisée, calcule le seuil dynamiquement en analysant l'histogramme de l'image. Il détermine mathématiquement la valeur qui maximise la variance inter-classe (la séparation optimale entre l'encre et le fond), garantissant un résultat robuste indépendamment de l'exposition initiale. (Voir fonction `otsu_maison`).

Normalisation topologique (Padding)

Un redimensionnement brut d'une image rectangulaire vers un carré écraserait les proportions du tracé (ex : un "1" très fin). La méthode du Padding ajoute des bordures blanches sur la plus petite dimension pour conserver strictement la géométrie. Une marge proportionnelle d'un sixième est ajoutée pour respecter les standards visuels de la base MNIST. (Voir fonction `rendre_image_carree`).

Formatage tensoriel 28x28

Après une rotation interactive corrigeant les métadonnées, la matrice est réduite à 28×28 pixels, puis aplatie en un vecteur de 784 valeurs. Les intensités sont inversées et normalisées via la formule $|x - 255|/255$. L'encre devient un signal d'activation fort (1) et le papier un signal nul (0), condition requise pour les neurones de la couche d'entrée.

Le réseau de neurones développé dans ce projet repose sur des concepts initiés dans la seconde moitié du XXe siècle. Aujourd'hui, ces modèles ont évolué vers des architectures **génératives** massives (ChatGPT, Claude, Gemini, Mistral). Si ces avancées technologiques sont majeures, elles soulèvent de profonds enjeux socio-écologiques.

Le coût énergétique et matériel

L'entraînement de modèles requiert l'analyse de pétaoctets de données et une puissance de calcul colossale. L'usage quotidien de ChatGPT mobiliserait ainsi près de 29 000 processeurs graphiques interconnectés. Dans son ouvrage *Contre-atlas de l'intelligence artificielle*, la chercheuse Kate Crawford souligne qu'OpenAI estime que la puissance de calcul allouée à l'IA est multipliée par dix chaque année depuis 2012. La consommation de ChatGPT est évaluée à 564 MWh par jour, soit l'équivalent électrique de dizaines de milliers de foyers.

L'impact environnemental des infrastructures

Au-delà de la consommation électrique, l'empreinte matérielle et hydrique est alarmante. Le refroidissement des centres de données nécessite d'importantes quantités d'eau douce, estimées à un demi-litre par simple conversation. Parallèlement, la fabrication des GPU exige l'extraction massive de métaux et de terres rares (cuivre, aluminium, cobalt, néodyme). Ces exploitations minières emploient des produits chimiques toxiques, génèrent des résidus radioactifs et contribuent à la dégradation durable des écosystèmes locaux.

Les bouleversements sociétaux et professionnels

Sur le plan social, le déploiement de l'IA bouleverse le marché du travail. Historiquement restreinte à l'automatisation mécanique (tri du courrier), l'intelligence artificielle investit désormais les secteurs cognitifs et créatifs (traduction, génération de contenu). Les modèles actuels sont entraînés pour piloter des véhicules ou pour assister, voire remplacer, du personnel soignant et des professeurs. Des services entiers sont déjà supplantés par des algorithmes, notamment dans la Silicon Valley. Cette transition foudroyante soulève de lourdes questions éthiques quant à l'obsolescence de certaines professions et à notre dépendance technologique.

Mise en perspective avec notre projet

À l'échelle de notre étude, le développement de notre propre perceptron multicouche nous a directement confrontés à ces limites de ressources. Les temps d'exécution prohibitifs rencontrés initialement nous ont obligés à repenser notre architecture. Le travail d'optimisation détaillé dans ce rapport démontre qu'une démarche de sobriété numérique est réalisable. Comprendre mathématiquement la descente de gradient permet non seulement de concevoir des IA, mais surtout de les rendre matériellement et énergétiquement plus efficaces.

■ ANALYSE RÉFLEXIVE DU PROJET

Réalisé sur dix semaines lors de notre quatrième semestre, ce projet a nécessité plus de quarante heures de travail en groupe. Cette analyse dresse le bilan méthodologique, technique et humain lié à la conception intégrale d'un programme d'intelligence artificielle par une équipe de cinq étudiants.

Définition des objectifs

Nous avons débuté notre immersion par l'étude de la descente de gradient, codant un premier script Python pour en saisir les mécanismes fondamentaux. Très vite, nous avons visé un objectif plus ambitieux : créer de toutes pièces un réseau de neurones multicouches capable de reconnaître des caractères manuscrits via le jeu EMNIST. Ce parcours a été rythmé par des réunions régulières avec notre tuteur. Son regard bienveillant a été crucial, particulièrement lorsque nous ne savions pas par où commencer le code ; il a su valider nos acquis théoriques tout en nous guidant dans nos avancées concrètes.

Une organisation d'équipe dynamique

Pour gérer cette charge de travail, nous avons privilégié une organisation souple plutôt que des rôles figés. L'atelier réflexif a d'ailleurs marqué un tournant majeur : ce temps d'arrêt nous a permis de faire le point sur notre progression et de relancer une dynamique en autonomie, rendant le travail hebdomadaire de plus en plus conséquent. En complément, les bilans réguliers sur Moodle ont servi de boussole interne au groupe. Ils nous ont permis de mieux nous situer les uns par rapport aux autres et de comprendre les ressentis de chacun, garantissant ainsi une cohésion nécessaire pour avancer sereinement.

Optimisation et tests croisés

Traduire la théorie en algorithmes a constitué notre principal défi. Nous avons adopté une stratégie incrémentale : partir d'une simple régression linéaire pour évoluer vers un modèle à deux, puis quatre neurones, augmentant la complexité au rythme de notre compréhension. Face aux limites de calcul de nos machines, un effort important a été consacré à l'optimisation du code. En parallèle, la création d'un script de conversion d'images en matrices nous a permis de tester la fiabilité du réseau sur nos propres données, tout en assurant une rédaction documentaire continue du rapport.

Conclusion

En définitive, ce projet de quatrième semestre s'est révélé extrêmement formateur pour tout le monde. Sur le plan technique et humain, il nous a permis de démystifier le machine learning en passant d'une théorie abstraite à une réussite collective concrète et optimisée.

■ BILAN DE L'IMPLÉMENTATION

Ce projet s'est attaché à démystifier les fondements de l'apprentissage profond. Face à la complexité des modèles modernes, souvent perçus comme des « boîtes noires », notre démarche a consisté à concevoir intégralement un perceptron multicouche. En nous affranchissant des bibliothèques de haut niveau au profit d'un calcul matriciel strict via NumPy, nous avons démontré qu'il est possible d'implémenter rigoureusement la descente de gradient et la rétropropagation pour aboutir à un modèle de reconnaissance de caractères (EMNIST) performant.

L'optimisation au cœur de la performance

Ce travail de fond a mis en évidence une réalité fondamentale : l'efficacité d'une intelligence artificielle ne repose pas uniquement sur le volume de données ingérées, mais intrinsèquement sur la qualité de son optimisation mathématique. La gestion minutieuse des hyperparamètres (taux d'apprentissage, initialisation) et l'optimisation matérielle des opérations matricielles se sont révélées être les véritables moteurs de la convergence du modèle.

Limites et architectures spécialisées

Si le perceptron multicouche constitue un socle théorique indispensable, son architecture « dense » montre des limites face à des données hautement dimensionnelles. Cette contrainte ouvre naturellement la voie à des modèles plus spécialisés. Les Réseaux de Neurones Convolutifs (CNN) s'imposent pour le traitement d'images complexes en exploitant les invariances spatiales, tandis que les modèles *Transformers* et les Réseaux Récurrents (RNN) dominent l'analyse séquentielle et le langage naturel. L'apprentissage profond forme ainsi un vaste écosystème d'architectures adaptées à la topologie de chaque donnée.

Un nouvel instrument pour la recherche scientifique

C'est précisément dans l'exploitation de ces modèles avancés que se dessinent les applications les plus prometteuses, bien au-delà des usages grand public. L'apprentissage profond s'impose désormais comme un outil scientifique de rupture : il permet de modéliser instantanément la dynamique des fluides, de prédire le repliement des protéines en biologie structurale (à l'image des travaux révolutionnaires d'[AlphaFold](#)), ou encore d'accélérer la découverte de nouveaux matériaux pour la transition énergétique. Comprendre le fonctionnement d'un perceptron, c'est finalement acquérir le vocabulaire de base d'une révolution scientifique en marche.

■ CARNET DE BORD ET COMPTES RENDUS D'AVANCEMENT

Ce journal de bord retrace les étapes clés de la gestion du projet, de la constitution de l'équipe à l'analyse finale des performances du réseau de neurones.

Date	Objectifs de la séance	Réalisations	Remarques du tuteur
29/01	Constitution du groupe et définition des outils.	Mise en place de l'espace Overleaf et des canaux de partage.	-
05/02	Initiation à la descente de gradient.	Programmation d'un algorithme rudimentaire (Batch) en Python.	-
12/02	Approfondissement théorique.	Étude des ressources pédagogiques (3Blue1Brown) sur les réseaux de neurones.	-
19/02	Réunion de suivi : Architecture et scénario.	Implémentation du mini-batch. Début du script de traitement d'images (<i>scenario.py</i>).	Proposition d'une étude de régression linéaire dans $\mathbb{R}$. Validation de la binarisation.
05/03	Séance de réflexivité et redistribution.	Analyse des blocages. Répartition des tâches (rétropropagation, TES, structure MLP).	-
19/03	Réunion de suivi : Optimisation et rapport.	Finalisation de la règle de la chaîne. Définition du plan du rapport (Démonstration → Théorie).	Conseil : comparer la règle de la chaîne au taux d'accroissement. Utiliser un <i>toy model</i> pédagogique.
26/03	Optimisation et vectorisation.	Passage des boucles <i>for</i> aux opérations matricielles (NumPy). Rédaction sur Overleaf.	-
09/04	Bilan final et performances.	Analyse du goulot d'étranglement. Taux de réussite atteint : 85,2 % sur EMNIST.]	Recommandation : réduire la taille du rapport et privilégier la généralisation des idées.

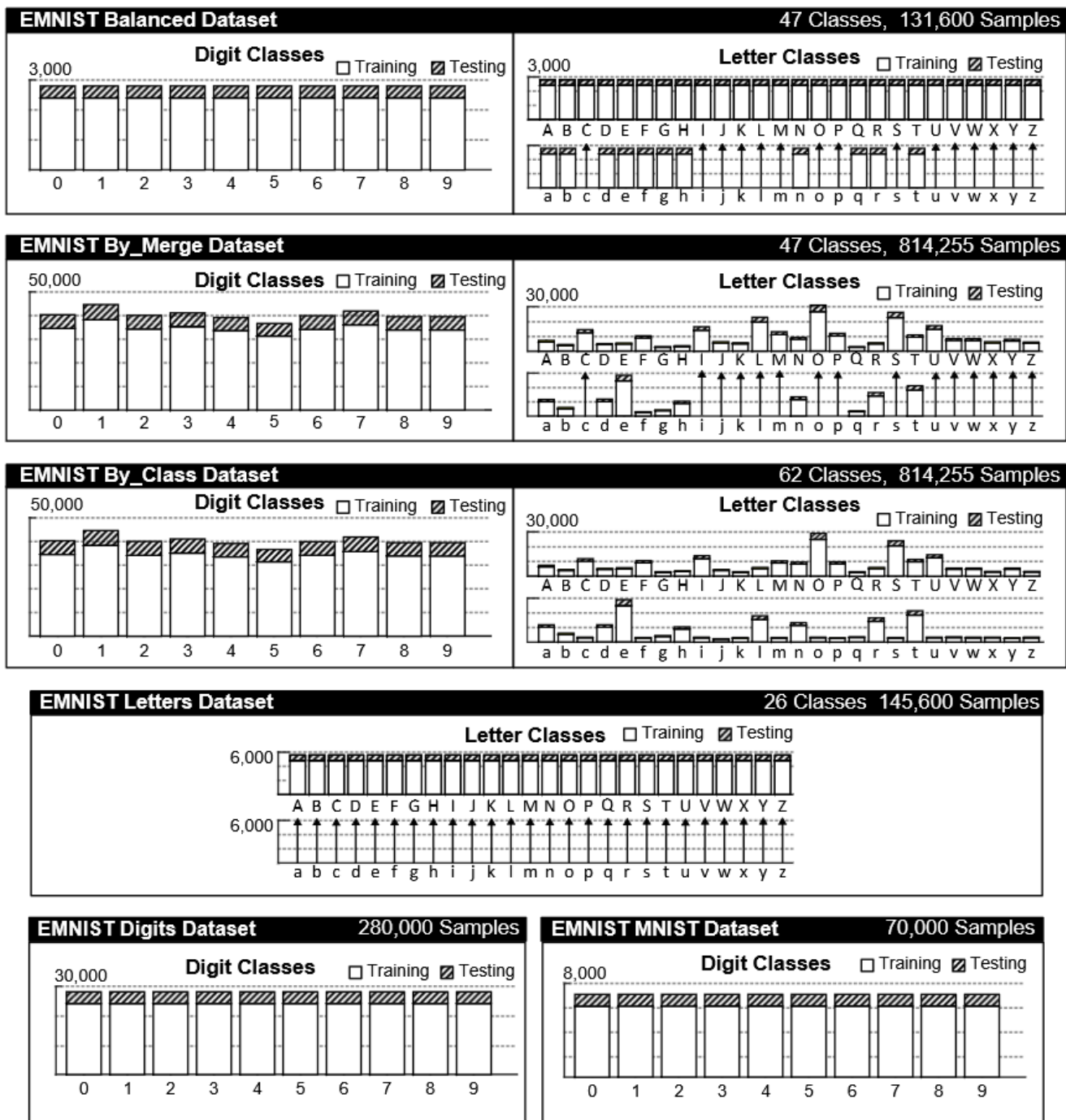


FIGURE 3.1 – Visualisation de la segmentation du jeu de données EMNIST

La base de données EMNIST inclut une classification optimisée qui regroupe notamment les lettres majuscules et minuscules à trop forte ressemblance (o/O, c/C, ...) (Exemples : segmentations « Balanced » ou « By Merge »). Cette segmentation permet d'entraîner le réseau de neurones sur des tâches précises et d'optimiser ses performances lorsqu'il est affecté à ces tâches.

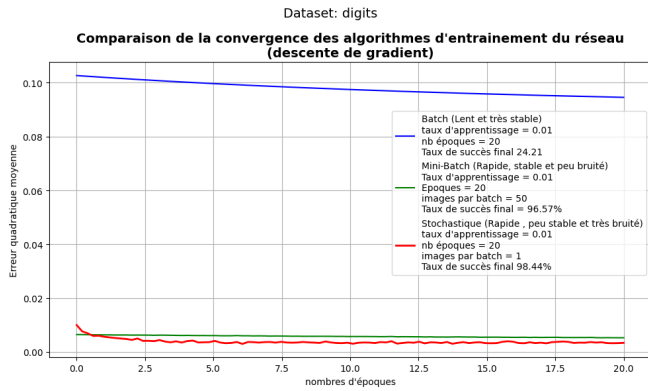


FIGURE 3.2

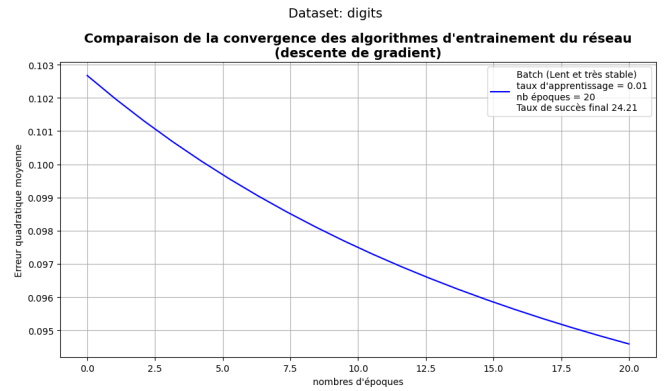


FIGURE 3.3

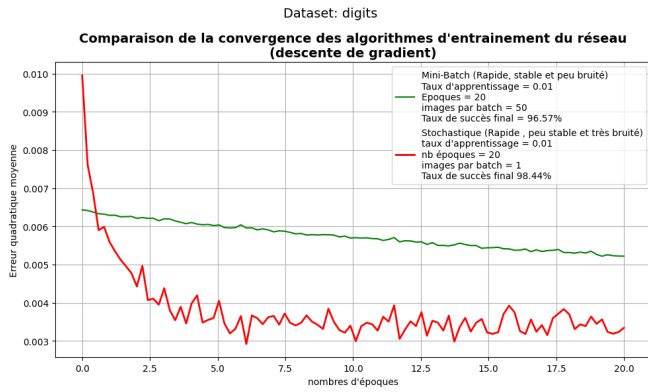


FIGURE 3.4

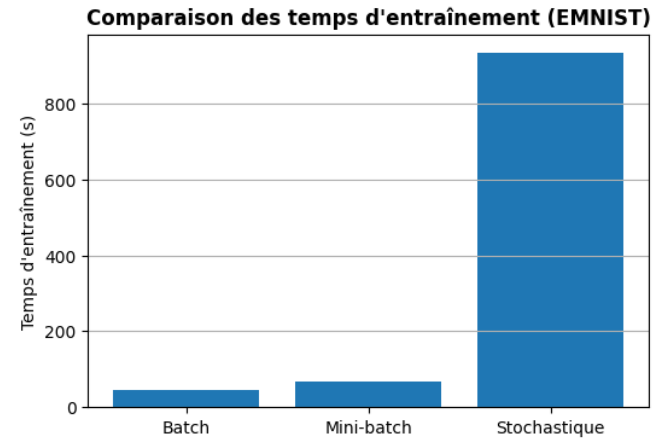


FIGURE 3.5

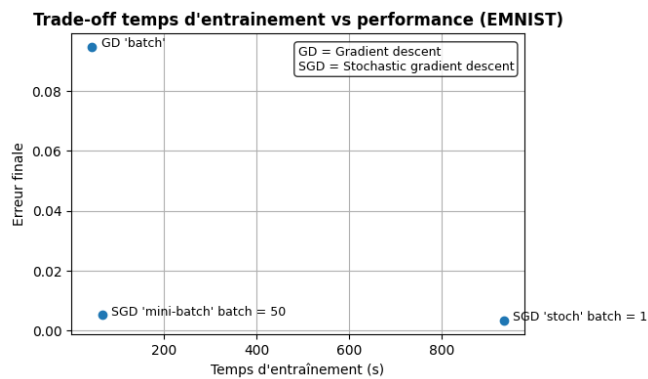


FIGURE 3.6

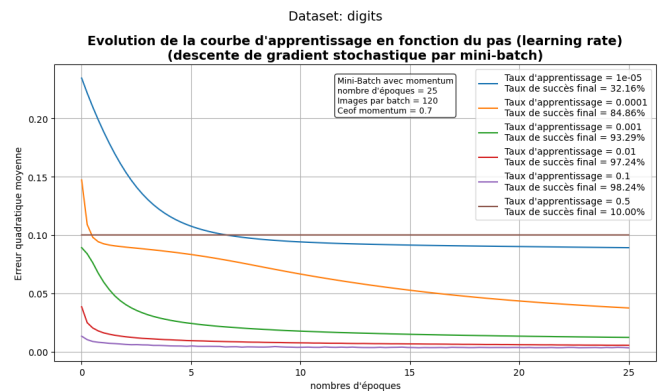


FIGURE 3.7

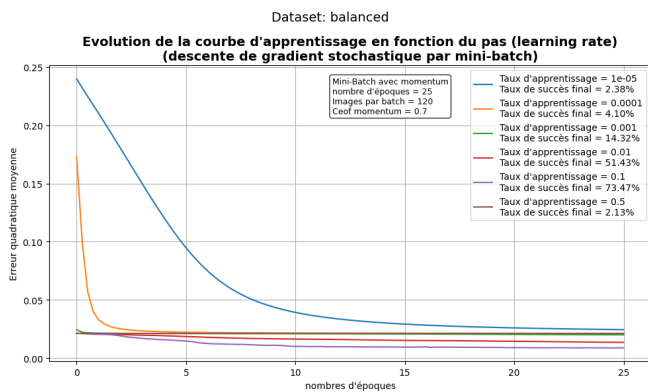


FIGURE 3.8

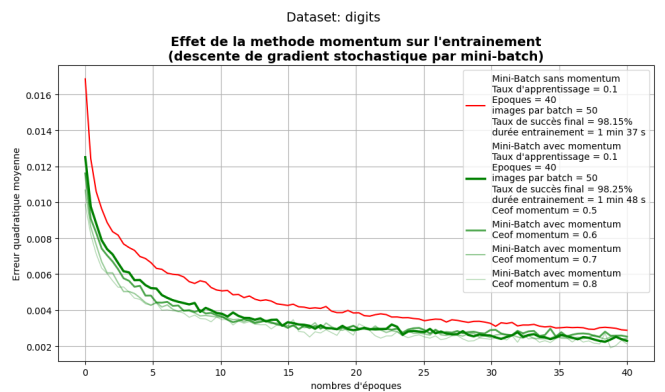


FIGURE 3.9

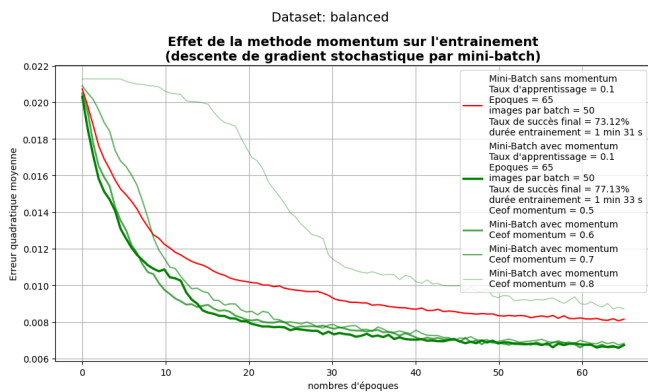


FIGURE 3.10

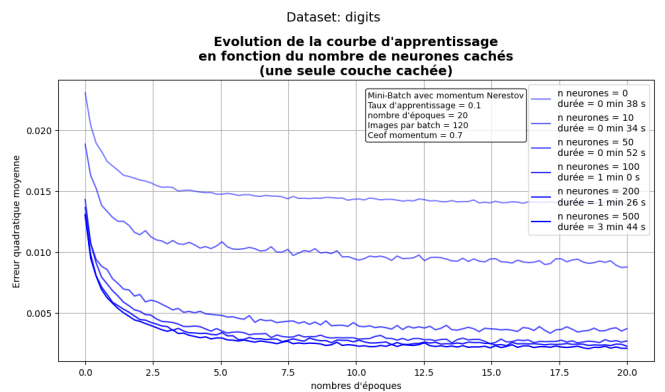


FIGURE 3.11

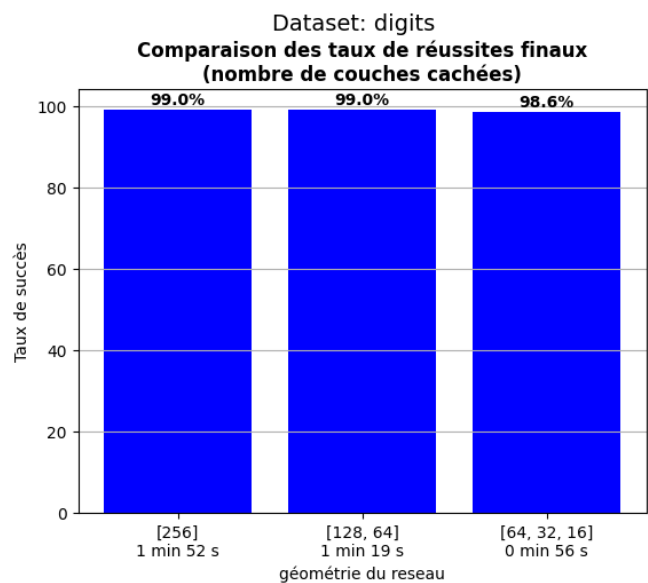


FIGURE 3.12

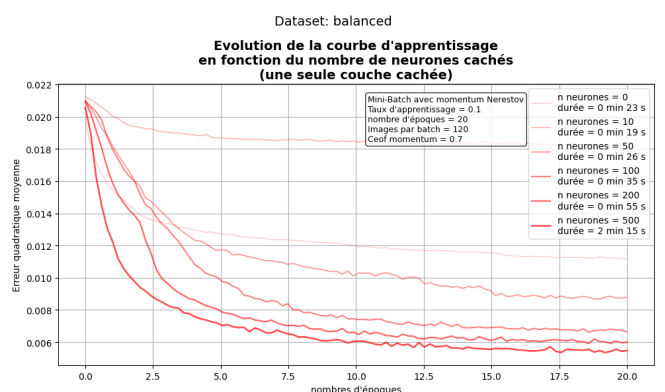


FIGURE 3.13

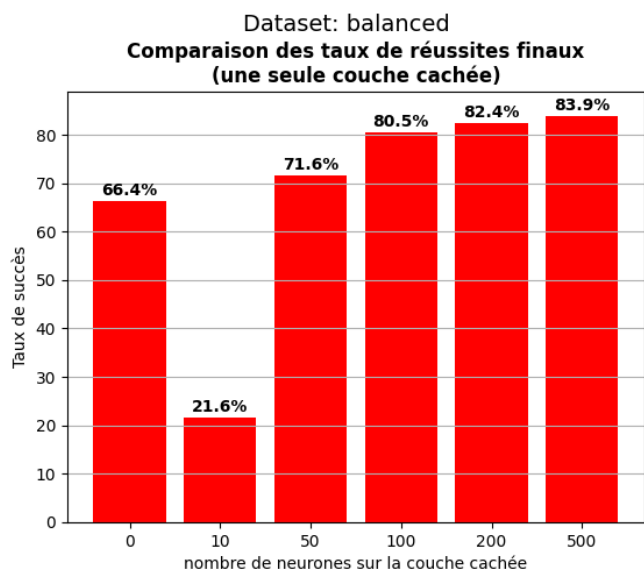


FIGURE 3.14

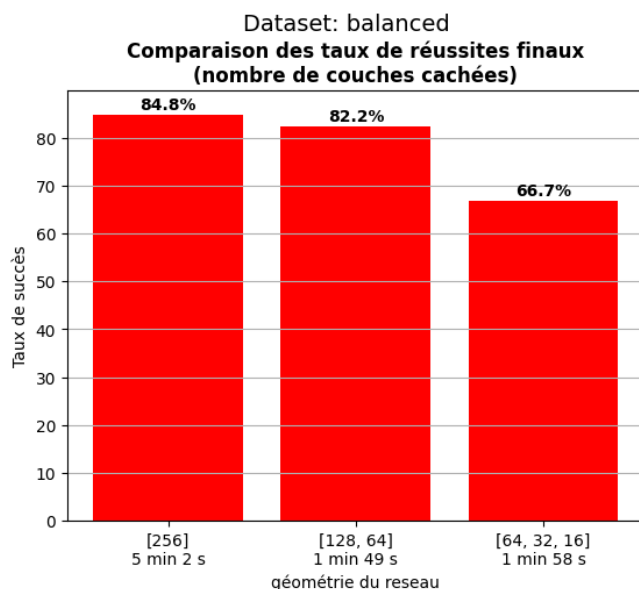


FIGURE 3.15

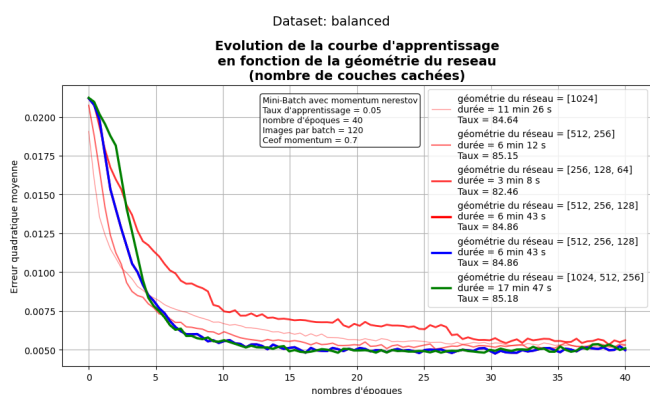


FIGURE 3.16

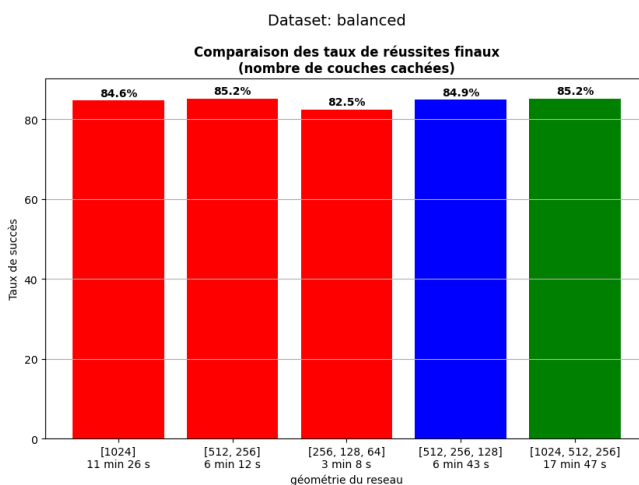


FIGURE 3.17

Exemples d'échec du réseau
géométrie : [256, 128, 64]
Entraînement : mini_batch_avec_momentum_nerestov
Dataset : balanced
Label : 5 réseau : S

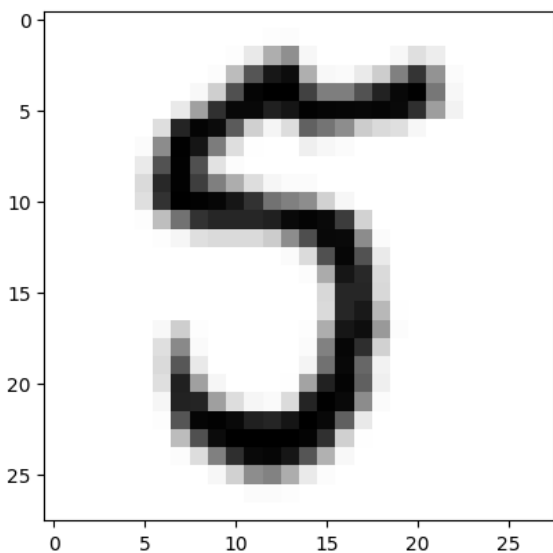


FIGURE 3.18

Exemples de succès du réseau
géométrie : [512, 256]
Entraînement : mini_batch_avec_momentum_nerestov
Dataset : balanced
Label : a réseau : a

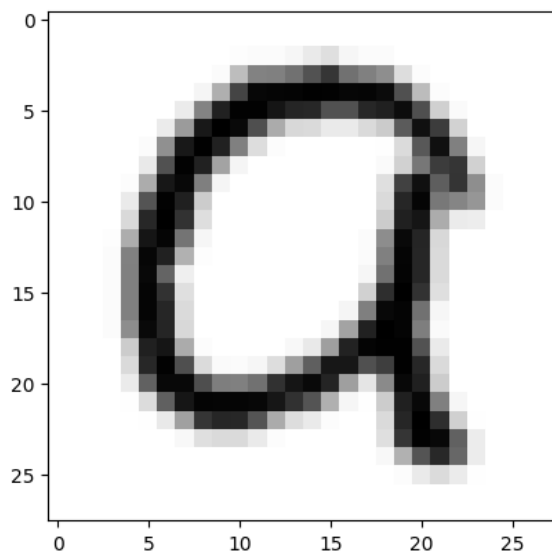


FIGURE 3.19

- [1] 3BLUE1BROWN. *Backpropagation calculus* | *Deep Learning Chapter 4*. Vidéo Youtube. 2017. URL : <https://www.youtube.com/watch?v=tIeHLnjs5U8>.
- [2] 3BLUE1BROWN. *Backpropagation, intuitively* | *Deep Learning Chapter 3*. Vidéo Youtube. 2017. URL : <https://www.youtube.com/watch?v=Ilg3gGewQ5U>.
- [3] Gregory COHEN et al. *EMNIST : an extension of MNIST to handwritten letters*. 2017. arXiv : 1702.05373 [cs.CV].
- [4] Ian GOODFELLOW, Yoshua BENGIO et Aaron COURVILLE. « Deep Learning ». In : MIT Press, 2016. Chap. 8 : Optimization for Training Deep Models. URL : <http://www.deeplearningbook.org>.
- [5] IBM. *Qu'est-ce qu'un réseau neuronal ?* Consulté le 3 avril 2026. 2024. URL : <https://www.ibm.com/fr-fr/think/topics/neural-networks>.
- [6] Memo JIMENEZ. *Rho-class : A custom LaTeX class*. Code source adapté pour la mise en forme personnalisée de l'environnement abstract. URL : <https://github.com/MemoJimenez/Rho-class> (visité le 06/04/2026).
- [7] Yann LECUN et al. « Efficient BackProp ». In : *Neural Networks : Tricks of the Trade*. Sous la dir. de Genevieve B. ORR et Klaus-Robert MÜLLER. Springer, 1998.
- [8] Yann LECUN et al. « Gradient-Based Learning Applied to Document Recognition ». In : *Proceedings of the IEEE* (1998). URL : <https://ieeexplore.ieee.org/document/726791>.
- [9] Fangfang LEE. *Que sont les réseaux neuronaux ?* Rapp. tech. IBM, 2026.
- [10] Marc PARIZEAU. *Réseaux de neurones : GIF-21140 et GIF-64326*. pdf, 2004.
- [11] Sebastian RUDER. « An overview of gradient descent optimization algorithms ». In : *arXiv preprint arXiv :1609.04747* (2016). URL : <https://arxiv.org/abs/1609.04747>.
- [12] THE NUMPY DEVELOPERS. *NumPy reference — NumPy v2.4 Manual*. Consulté le 6 avril 2026. 2026. URL : <https://numpy.org/doc/2.4/reference/>.
- [13] Walter Pitts WARREN S. MCCULLOCH. « A logical calculus of the ideas immanent in nervous activity ». In : *Bulletin of mathematical biophysics, vol.5* (1943).
- [14] WIKIPÉDIA. *Perceptron multicouche*. 2024. URL : https://fr.wikipedia.org/wiki/Perceptron_multicouche.
- [15] WIKIPÉDIA. *Réseau de neurones artificiels*. 2024. URL : https://fr.wikipedia.org/wiki/R%C3%A9seau_de_neurones_artificiels.